

Automated Morphological Classification of Galaxies Using Deep Learning Techniques

January 13, 2025

1 Introduction

Classifying galaxies based on their appearance is an essential part of astrophysics. Galaxy morphology can inform us about the formation and evolution of the galaxy and its makeup. It is one of the most preliminary forms of assessment one can do before investing it further. Traditionally reliant on human expertise—such as the Hubble Sequence and Galaxy Zoo initiatives—the manual classification has become impractical due to the rapid growth of astronomical datasets, highlighting the need for automated methods.

2 Literature Review

For a very long time, galaxy morphology identification remained a human-led task and a ripe field for AI and ML techniques. The advent of traditional supervised learning methods such as SVM and Random Forests marked the beginning of automated classification in the field; for example, Banerji et al. (2010) utilized SVM to distinguish between early and late-stage galaxies. These methods suffered from the models' inability to infer the underlying spatial features.

Machine learning (ML) has become an integral part for automating the classification of galaxies from large imaging catalogs [1]. Convolutional Neural Networks (CNNs) are particularly effective, at capturing spatial hierarchies in images to detect subtle morphological features [5]. Recent advancements like Vision Transformers (ViTs) and hybrid CNN-ViT models further enhance the identification of complex patterns in image data [2].

However, there remain a number of challenges in this field, especially when it comes to the availability of data catalogs. One of the most common problems is a lack of data points or a significant imbalance in the labeled data[3]. To combat this, we explore the Few-Shot learning method in this study and some other architectures in this course to address this issue and see if this is a viable way out of this problem.

This project aims to categorize galaxies under the Hubble sequence [4] in the Galaxy10 dataset [6].

3 Data

The Galaxy10 data [6] set serves as the primary data set for this study. This data set is derived from the Galaxy Zoo initiative and consists of a curated collection of galaxy images from the Sloan Digital Sky Survey (SDSS). The galaxies within this collection are categorized into ten distinct classes. Due to the intricate nature of galaxy classification, each image is accompanied by a questionnaire answered by multiple participants. Only those images that achieve over 55% agreement among respondents are included in the final dataset. This data set needs to improve on problems such as class imbalance, as odd-shaped galaxies have a considerably lower number of data points relative to the other classes. In many instances, there isn't a clear distinction between classes, which is a sticking point for many machine learning models [6].

Three machine learning techniques were used in this study. Namely Convolution Neural networks(CNN), Visual Transformers (ViT) and the few shot method. Preparing the data for each of the models' training and testing is described below. The dataset is available through the astroNN library which was used in the code to import the data.

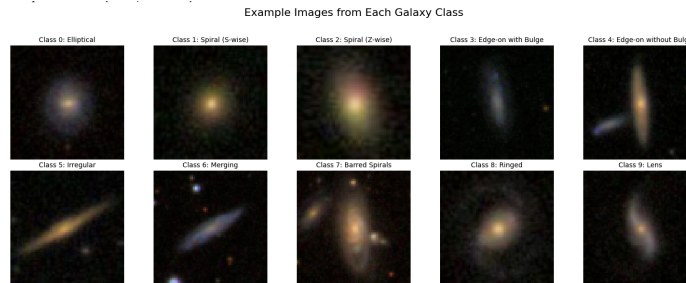


Figure 1: Different galaxy classes in the data set

3.1 CNN Data Preparation

The data was imported using the astroNN library through the loadgalaxy10sdss method. Each image was 69 by 69 pixels, an ideal size for ML algorithms. Each image was normalized in the $[0,1]$ range by scaling the pixel values by 255. This helps stabilize the training process and detect features. Labels were then one-hot encoded into ten categories. Some images were augmented to account for data imbalances, i.e., rotated or reflected about an axis to act as new data points. However, it was later found in the training that this made the data set too big. I ran out of system resources despite using Google Collab's T4 GPU, so this method was abandoned. However, given enough system resources, image augmentation remains widely used in astronomy ML research. The dataset was then split into an 80-20 Train test split, fed, and processed in batches of 64

images due to resource constraints.

3.2 ViT Data Preparation

The data was normalized, one-hot encoded and split to train-test in an identical manner to the CNN model. A further 10% of the training data was used as a validation set for model training.

3.3 Few-Shot Data Preparation

The data was loaded and normalized identically to the previous models. To prepare it for few-shot learning, the data was split into a support set and a query set, which had n number of images for each class. These sets are used to evaluate the model's prediction based on the computed prototypes. To handle class imbalance, if the number of data points in a particular class is less than $2n$, then the number of query and support samples is reduced to fit the available data.

4 Model Architecture and Training

Three models [CNN (2 different models), ViT, and few-shot methods] with varying architectures were used to find the best technique for classifying data in this set. Further, the hyper-parameters were varied for a given model to find the best version of the model. Below, we describe briefly each model's architecture and training method. It should be noted that each model was only trained for five epochs due to time and resource constraints.

4.1 Basic CNN Model

In the basic model, two convolution layers are used in succession, which feed into a max-pooling layer to reduce spatial dimensions and capture the dominant features of the image. Max-pooling also reduces the computational complexity and risk of overfitting the data. A pool size of 2x2 is used to halve the dimensions after each pooling operation. Further dropout layers are implemented to combat overfitting in the model. For the fully connected layers, the flattened feature map is first passed into a dense layer with 128 units. ReLu is used as the activation function to avoid vanishing gradients because it is a standard practice to use this function in CNNs. Then, the last dense layer has ten units corresponding to the ten classes, which uses the softmax activation function for probability distributions. The model is trained in 32 and 64-layer configurations with drop rates of 0.2 and 0.3, respectively. The model uses the Adam optimizer, which is efficient for deep NN training, and the categorical cross-entropy loss function, which is known to be suitable for multiclass classification problems.

4.2 Advanced CNN Model

The advanced model takes in the input of the form(69,69,3) instead of using a one-dimensional convolution and max-pooling layer. This model uses convolution blocks, each with a Conv2D [9], MaxPooling2D [11], and a BatchNormalization [8] layer. The Conv2D layer again uses a ReLU activation function as before, which was found to perform satisfactorily in the basic model. The MaxPooling2D reduces spatial dimensions by a factor of 0.5, which makes it computationally efficient, and the BatchNormalisation layer normalizes activations to improve stability and accelerate convergence of the model.

Global Average Pooling (GAP) [10] is utilized for feature extraction, which reduces each feature to a single value by averaging spatial information. The first FCN is a dense layer with 256 units and a dropout layer with a 50% rate. The output layer is identical to the basic model. The model uses the Adam optimizer, which is efficient for deep NN training, and the categorical cross-entropy loss function, which is known to be suitable for multiclass classification problems.

4.3 Visual Transformer

The model takes in input of the form(69,69,3). Using a convolution layer, the image is divided into non-overlapping patches. Positional embeddings are added to patch embeddings to encode spatial information. Next, multiple transformer encoder blocks are stacked, each containing a Multi-Head Self-Attention block to compute attention weights for every patch. Skip connections are added after self-attention and MLP layers for better gradient flow. The output sequence is flattened and passed to an FCN layer with ten outputs. The training consists of a grid search for the best hyperparameters, wherein patch size, embedding, number of heads, and number of layers are varied to find the best possible combination. The ADAM optimizer is used with a learning rate of 0.001.

4.4 Prototype Network/ Few-Shot

Prototypical Networks, introduced by Snell et al. in 2017 [7], serve as a framework for few-shot learning. These models aim to classify new instances using only a small number of labeled examples per class, utilizing the idea of prototype representations. For each class, a "prototype" is computed as the mean of the feature embeddings from its support set. Classification is conducted by comparing the query instance to these prototypes through a distance metric, typically using Euclidean distance. Prototypical Networks are effective in simplifying the task to learning a feature embedding space where samples of the same class group closely together, making them particularly suitable for low-data conditions.

The prototype network consists of feature extractors built around CNN. This network is designed to generate feature embeddings for input images, enabling effective representation of their visual characteristics. Most of the architecture

follows the basic CNN, wherein after passing through the FCN, we calculate the prototype vector for each class in the data set. This represents the center of the class in the embedding set, following which the embeddings of the query sample are calculated, and then the Euclidean distance between query embeddings and each class prototype is calculated. The softmax function then turns this into probabilities representing the likelihood of the query sample belonging to each class. The model is trained by reducing the categorical cross-entropy loss between the actual query labels and the predicted probabilities [7].

5 Results and Discussion

5.1 CNN Models

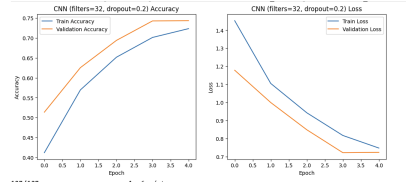


Figure 2: Accuracy and training loss for basic CNN

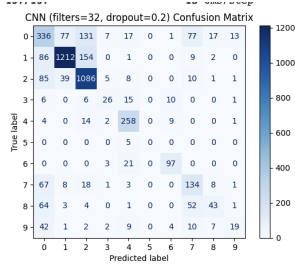


Figure 3: Confusion matrix for basic CNN

The model shows a steady improvement in both training and validation accuracy. The maximum training accuracy is 72%, while the maximum validation accuracy reached is 74%. This shows that the model generalizes well, indicating no overfitting. The reduction of training loss over epochs also indicates effective learning. The confusion matrix shows us that classes 2 and 1 are well-classified, with the highest count in the diagonal. The models struggle with smaller classes, such as 6 and 7. This is clearly due to the data imbalances between the classes and the need for distinctive features between the images.

Overall, the model demonstrates a steady improvement in learning. Images in classes 6 and 7 will benefit from data augmentation techniques due to their

smaller sample size. We also observe that the validation accuracy plateaus after epoch 4, suggesting that additional epochs may not improve the model much.

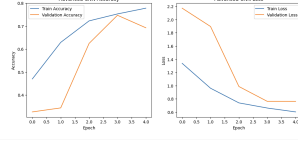


Figure 4: Accuracy and training loss for advanced CNN

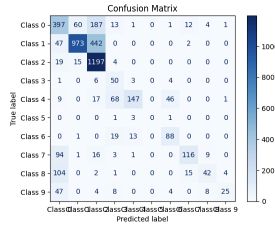


Figure 5: Confusion matrix for advanced CNN

The advanced model achieves 77.79% accuracy vs. 72.31% for the basic version but worsens in validation accuracy to 69.25% from 74.30%, indicating slight overfitting. Again, classes 1 and 2 are the best performers, while classes 0 and 9 show notable mis-classifications into other classes. Data imbalance still remains a problem, given the low accuracy in classes 6 and 7. It can be concluded that it is better to use the basic CNN model.

5.2 ViT Model

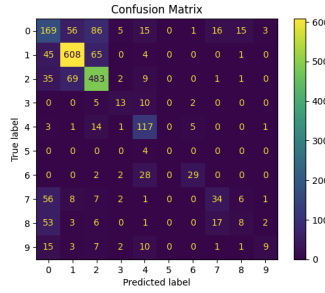


Figure 6: Transformer model confusion matrix

The best model performance is setting the patch size to 9, embed dom = 64, num heads = 2, and num layers = 3. This achieves an accuracy of 67.46%. It is observed that a higher embedding dimension leads to overfitting, while

increasing the number of heads does not improve the model accuracy, and deeper models achieve better accuracy but, again, are prone to overfitting.

5.3 Prototype Network

The few-shot model is the worst-performing model, with a mere 30% accuracy. It is best to avoid this approach. The model struggles because it needs to generalize using very few examples. The lack of clear distinction between the galaxies and the diverse classification range types makes the problem worse.

6 Conclusion

In conclusion, this study suggests that models with similar architecture to CNN should be considered for future galaxy classification and identification efforts. This is supported by other studies, such as [12], that use more advanced architectures, such as Alexnet and Resnet, for galaxy classification. Investigating more hybrid models also remains an interesting avenue.

Apart from that, class imbalance remains a major problem which should be addressed through the means of image augmentation.

Appendix

Patch Size	Embed Dim	Num Heads	Num Layers	Test Accuracy	Val Accuracy (Epoch
4	64	2	2	65.63%	66.34%
4	64	2	3	67.00%	66.96%
4	64	4	2	66.09%	66.50%
4	64	4	3	65.72%	65.22%
8	64	2	2	64.11%	65.12%
8	64	2	3	67.46%	67.67%
8	64	4	2	66.04%	67.98%
8	64	4	3	63.15%	63.95%
4	128	2	2	64.20%	64.25%
4	128	2	3	63.10%	63.59%
4	128	4	2	64.57%	66.50%
4	128	4	3	60.17%	59.36%
8	128	2	2	65.49%	66.85%
8	128	2	3	63.42%	63.59%
8	128	4	2	66.50%	67.57%
8	128	4	3	34.42%	33.10%

Table 1: Summary of Transformer Model Results with Different Configurations

References

- [1] S. B. Cho, S. Cousins-Baxter, S. Ruberto, and V. Terragni. Automated trustworthiness testing for machine learning classifiers. *ArXiv (Cornell University)*, 2024. doi: 10.48550/arxiv.2406.05251. URL <https://doi.org/10.48550/arxiv.2406.05251>.
- [2] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*, 2020.
- [3] Ibrahim Gönenc and Bilge Türe. Class imbalance in astronomical data and its implications. *Monthly Notices of the Royal Astronomical Society*, 493(1):579–588, 2020.
- [4] E. P. Hubble. Extra-galactic nebulae. *Contributions from the Mount Wilson Observatory / Carnegie Institution of Washington*, 324:1–49, 1926.
- [5] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. Imagenet classification with deep convolutional neural networks. *Advances in Neural Information Processing Systems*, 25, 2012.
- [6] Chris Lintott, Kevin Schawinski, Steven Bamford, Anže Slosar, Kate Land, Daniel Thomas, Edd Edmondson, Karen Masters, Robert C. Nichol, M. Jordan Raddick, Alex Szalay, Dan Andreescu, Phil Murray, and Jan Vandenberg. Galaxy zoo 1: data release of morphological classifications for nearly 900,000 galaxies. *Monthly Notices of the Royal Astronomical Society*, 410(1):166–178, January 2011. doi: 10.1111/j.1365-2966.2010.17432.x.
- [7] Jake Snell, Kevin Swersky, and Richard Zemel. Prototypical networks for few-shot learning. In *Advances in Neural Information Processing Systems*, 2017.
- [8] TensorFlow Developers. Tensorflow batch-normalization layer documentation, . URL https://www.tensorflow.org/api_docs/python/tf/keras/layers/BatchNormalization. Accessed : 2024 – 12 – 01.
- [9] TensorFlow Developers. Tensorflow conv2d layer documentation, . URL https://www.tensorflow.org/api_docs/python/tf/keras/layers/Conv2D. Accessed : 2024 – 12 – 01.
- [10] TensorFlow Developers. Tensorflow globalaveragepooling2d layer documentation, . URL https://www.tensorflow.org/api_docs/python/tf/keras/layers/GlobalAveragePooling2D. Accessed : 2024 – 12 – 01.

- [11] TensorFlow Developers. Tensorflow max-pooling2d layer documentation, . URL https://www.tensorflow.org/api_docs/python/tf/keras/layers/MaxPooling2D. Accessed : 2024 – 12 – 01.
- [12] Authors Unknown. Galaxy morphology classification with densenet. In *Journal of Physics: Conference Series*, volume 2402, page 012009. IOP Publishing, 2022. doi: 10.1088/1742-6596/2402/1/012009.